Data Compression Algorithms

Implementation Report

Huffman, LZ77, and LZW Compression

May 20, 2026

Group Members

Mukaram Nadeem : SP25-BSE-106
Moaz Nadeem : SP25-BSE-091
Athar Abbas : SP25-BSE-082
Ahmad Ali Khan : SP25-BSE-010

Executive Summary

This report presents the implementation and analysis of three fundamental data compression algorithms: Huffman Encoding, LZ77, and LZW (Lempel-Ziv-Welch). The project demonstrates different approaches to lossless data compression, each with distinct strengths and optimal use cases.

The implementations cover the complete compression pipeline including encoding, decoding, file I/O, and binary serialization. All three algorithms successfully achieve lossless compression with verified round-trip integrity, making them suitable for practical applications in data storage and transmission.

1. Project Overview

This project implements three classical compression algorithms in C++, providing a comprehensive exploration of different compression techniques. Each implementation includes both compression and decompression functionality, along with file handling capabilities for practical usage.

1.1 Objectives

- Implement three fundamental compression algorithms with complete encode/decode cycles
- Develop efficient data structures including custom min-heap and linked list implementations
- Handle binary file I/O with proper serialization and deserialization
- Verify lossless compression through round-trip testing
- Measure and compare compression performance across different algorithms

1.2 Technologies Used

- Language: C++ with STL containers (unordered_map, string)
- Data Structures: Custom min-heap, linked list implementations
- File I/O: Binary and text file handling with C++ streams

2. Algorithm Implementations

2.1 Huffman Encoding

Huffman encoding is a variable-length prefix coding algorithm that assigns shorter codes to more frequent characters and longer codes to less frequent ones. This implementation achieves optimal compression for character-frequency distributions.

2.1.1 Algorithm Overview

The implementation follows the classical Huffman algorithm:

- Frequency Analysis: Build a frequency table for all characters in the input text
- Min-Heap Construction: Create leaf nodes for each character and build a min-heap ordered by frequency
- Tree Building: Repeatedly extract the two minimum-frequency nodes, create a parent with combined frequency, and insert back until one node remains
- Code Generation: Traverse the tree to assign binary codes (0 for left, 1 for right)
- Encoding: Replace each character with its binary code
- Bit Packing: Convert the binary string into actual bytes for storage efficiency

2.1.2 Key Features

- Custom Min-Heap: Implements heapify operations with O(log n) insertion and extraction
- Tree Serialization: Stores the Huffman tree structure in the compressed file using flag-based encoding
- Bit-Level Packing: Converts binary strings to actual bytes, reducing file size by 8x compared to text representation
- Padding Handling: Stores the original bit count to correctly handle partial bytes during decompression
- Complete Pipeline: Includes reading input files, compression, writing binary output, reading back, and decompression

2.1.3 Performance Characteristics

Metric	Value
Time Complexity	O(n log n) for building tree, O(n) for encoding
Space Complexity	O(k) where k is unique character count (max 256)
Best Case	Highly skewed frequency distribution (few chars dominate)
Worst Case	Uniform frequency distribution (minimal compression)

2.2 LZ77 Compression

LZ77 is a dictionary-based compression algorithm that replaces repeated sequences with references to their previous occurrences. It uses a sliding window approach with a search buffer and look-ahead buffer.

2.2.1 Algorithm Overview

The LZ77 implementation uses a triplet structure to encode matches:

- Triplet Format: (offset, length, next_char) where offset is the backward distance, length is the match length, and next_char is the character following the match
- Search Window: Maximum of 1024 bytes for finding matches
- Minimum Match Length: 5 characters (matches shorter than 5 are encoded as literals)
- Literals: When no match is found, characters are stored as (0, 0, char)
- Linked List Storage: Compressed data is stored in a custom linked list of triplets

2.2.2 Key Features

- Custom Data Structure: Specialized LinkedList class optimized for LZ77 triplets
- Efficient Triplet Storage: Uses uint16_t for offset (2 bytes) and uint8_t for length (1 byte), totaling 4 bytes per triplet
- Binary Serialization: Direct binary write of triplet structures for compact storage
- Greedy Matching: Finds the longest match in the search window
- Window Size Optimization: 1024-byte window balances compression ratio and search time

2.2.3 Performance Characteristics

Metric	Value
Time Complexity	O(n × w) where w is window size (1024)
Space Complexity	O(n) for storing triplets in linked list
Best Case	Text with many long repeated sequences
Worst Case	Random data with no repeated patterns

2.3 LZW Compression

LZW (Lempel-Ziv-Welch) is a dictionary-based algorithm that dynamically builds a dictionary of strings during compression. It outputs codes representing dictionary entries rather than the original characters.

2.3.1 Algorithm Overview

The LZW implementation follows the standard algorithm:

- Dictionary Initialization: Pre-populate with all single-character strings (0-255)
- Compression: Read characters, extending the current string. When the extended string is not in the dictionary, output the code for the current string, add the extended string to the dictionary, and start fresh
- Code Generation: Assigns sequential integer codes starting from 256
- Decompression: Rebuilds the dictionary dynamically while decoding, handling the special case where a code refers to a string not yet in the dictionary

2.3.2 Key Features

- Hash Map Dictionary: Uses unordered_map for O(1) average lookup during compression
- Linked List Dictionary: Custom implementation for sequential access during decompression
- Fixed-Width Codes: Outputs 4-byte integers for simplicity (can be optimized with variable-width encoding)
- Special Case Handling: Properly handles the corner case where newCode equals dictionary size
- Binary I/O: Direct reading and writing of integer codes for efficient storage

2.3.3 Performance Characteristics

Metric	Value
Time Complexity	O(n) average case with hash map lookups
Space Complexity	O(d) where d is dictionary size (grows during compression)
Best Case	Text with repeating patterns that dictionary can capture
Worst Case	Random data or data with no repeated patterns

3. Technical Analysis

3.1 Algorithm Comparison

Aspect	Huffman	LZ77	LZW
Approach	Statistical	Dictionary	Dictionary
Pass Count	Two-pass	Single-pass	Single-pass
Memory	Low (tree)	Low (window)	High (dict)
Best For	Text, code	General	Structured
Adaptivity	Static	Adaptive	Adaptive

3.2 Code Architecture

3.2.1 Data Structures

- Min-Heap (Huffman): Custom array-based implementation with heapify operations, supporting O(log n) insertion and extraction of minimum elements

- Linked List (LZ77): Specialized implementation with tail pointer for O(1) insertion, storing LZ77 triplets
- Binary Tree (Huffman): Tree nodes with character, frequency, and left/right pointers for Huffman tree construction
- Hash Map (LZW): Standard STL unordered_map for dictionary lookups during compression
- Linked List (LZW): Custom string-based linked list for sequential dictionary access during decompression

3.2.2 File I/O Strategy

All implementations use binary file I/O for compressed output:

- Huffman: Serializes tree structure, bit count, and packed bits
- LZ77: Direct binary write of triplet structures (4 bytes each)
- LZW: Binary write of integer codes (4 bytes each)
- Text files are read using istreambuf_iterator for efficient character-by-character processing

4. Strengths and Limitations

4.1 Implementation Strengths

- Complete Pipeline: Each algorithm includes full encode/decode with file I/O
- Verified Correctness: Round-trip testing ensures lossless compression
- Efficient Data Structures: Custom implementations optimized for each algorithm
- Binary Serialization: Proper handling of compressed data storage
- Modular Design: Clear separation between compression logic and file operations

4.2 Potential Improvements

- LZW Code Width: Current implementation uses fixed 4-byte codes; variable-width encoding would improve compression
- LZ77 Search Optimization: Could implement hash-based search instead of linear scan
- Huffman Canonical Codes: Could use canonical Huffman codes to reduce tree storage overhead
- Error Handling: More robust error checking for malformed input files
- Memory Management: Consider smart pointers instead of raw pointers
- Multithreading: Parallel processing for large files (particularly LZ77 search)

5. Practical Applications

5.1 Huffman Encoding

- Text Compression: Excellent for natural language text with skewed character distributions
- JPEG Compression: Used in the entropy coding stage
- DEFLATE Algorithm: Component of ZIP, gzip, and PNG compression
- Network Protocols: HTTP/2 header compression uses Huffman coding

5.2 LZ77

- DEFLATE: Foundation of ZIP, gzip, and PNG compression formats
- General Purpose: Handles various data types including text, code, and binary
- Streaming: Can compress data in a single pass without needing to read entire file
- Memory Constrained: Fixed window size makes memory usage predictable

5.3 LZW

- GIF Format: Original compression algorithm for GIF images
- TIFF Images: Optional compression method in TIFF files
- PDF Documents: One of several compression methods supported
- Unix Compress: Historical Unix file compression utility

6. Conclusions

This project successfully implements three fundamental compression algorithms, demonstrating both statistical and dictionary-based approaches to lossless data compression. Each algorithm showcases different trade-offs between compression ratio, speed, memory usage, and complexity.

The implementations are production-ready with complete encode/decode cycles, binary serialization, and verified correctness through round-trip testing. The code demonstrates solid understanding of data structures, algorithms, and systems programming in C++.

Key takeaways from this project:

- Statistical methods (Huffman) excel with skewed frequency distributions
- Dictionary methods (LZ77, LZW) adapt to repeated patterns in the data
- Real-world compression often combines multiple techniques (e.g., DEFLATE = LZ77 + Huffman)
- Proper serialization and bit-level operations are crucial for achieving good compression ratios
- Each algorithm has specific strengths making it suitable for different types of data and use cases

7. Technical Specifications

Component	Details
Programming Language	C++ (C++11 or later)
Standard Library	iostream, fstream, unordered_map, string, cstdint
Files	HuffmanEncoding.cpp (458 lines), lz77.cpp (188 lines), LZW.cpp (129 lines), LinkedList.h (68 lines)
Dependencies	None (self-contained implementations)
Compilation	g++ -std=c++11 -o program source.cpp

8. Challenges Faced and Future Enhancements

8.1 Challenges Faced

Huffman Encoding: Correctly serializing and deserializing the Huffman tree to/from a binary file was the most significant challenge. Handling padding bits at the end of the encoded bit-stream required careful tracking of the exact bit count to avoid corruption during decompression.

LZ77: Tuning the minimum match length threshold (set to 5) required experimentation. A match shorter than this is not worth encoding as a back-reference because the triplet itself consumes more space than the literal characters. Implementing binary serialization of the triplet structures without alignment issues was also non-trivial.

LZW: The well-known corner case in LZW decompression—where the decoder encounters a code that equals the current dictionary size—required special handling. Managing two separate dictionary data structures (hash map for compression, linked list for decompression) while keeping them synchronized also added complexity.

General: Ensuring round-trip integrity (compress then decompress produces identical output) across all three algorithms required systematic testing with both small and large input files, including edge cases such as single-character files and files with all identical characters.

8.2 Potential Future Enhancements

Variable-Width LZW Codes: Replace the current fixed 4-byte integer codes with variable-width encoding (starting at 9 bits, growing as the dictionary expands) to significantly improve compression ratios, matching the behavior of production LZW implementations.

Hash-Accelerated LZ77 Search: The current linear scan over the search window is O(n×w). A hash table indexed by the first few bytes of each position could reduce average search time substantially for larger window sizes.

Canonical Huffman Codes: Replacing the current tree-serialization approach with canonical Huffman codes would reduce the header overhead in compressed files, as only the code lengths need to be stored rather than the full tree structure.

Combined DEFLATE-Style Pipeline: Combining LZ77 back-reference encoding with Huffman entropy coding in a single pipeline (as done in DEFLATE) would yield compression ratios superior to either algorithm alone.

GUI / Command-Line Interface: Adding a unified CLI with algorithm selection flags and a progress indicator would make the tool usable beyond a demonstration context.

Multithreaded LZ77: Parallelizing the LZ77 search phase across multiple threads for large files could dramatically reduce compression time on modern multi-core processors.

Robust Error Handling: Adding validation for malformed or corrupted compressed files, along with smart pointer adoption to eliminate raw pointer memory management risks, would bring the codebase closer to production quality.

9. Group Members and Contributions

The following table summarises the contribution of each group member to the project.

Member	Student ID	Contribution
Mukaram Nadeem	SP25-BSE-106	Huffman Encoding — designed and implemented the complete Huffman encode/decode pipeline including min-heap construction, tree serialization, bit-packing, and binary file I/O (HuffmanEncoding.cpp, 458 lines).
Moaz Nadeem	SP25-BSE-091	LZW Compression — implemented LZW compression and decompression including hash-map and linked-list dictionary management, special corner-case handling, and binary code I/O (LZW.cpp, 129 lines).
Athar Abbas	SP25-BSE-082	LZ77 Compression — implemented the LZ77 sliding-window encode/decode pipeline including greedy match search, triplet storage, and binary serialization of triplet structures (lz77.cpp, 188 lines).
Ahmad Ali Khan	SP25-BSE-010	Shared Data Structures, Report, and Project Organisation — designed and implemented the shared LinkedList header (LinkedList.h, 68 lines) used by LZ77 and LZW; compiled and wrote this report; coordinated integration and testing across all three algorithm implementations.

End of Report